Universidad Tecnológica de Honduras

ASIGNATURA:
PROGRAMACION MOVIL I

TRABAJO:
Tarea 2.3

CATEDRÁTICO:
Ricardo Lagos

REALIZADO POR:

José Fernando Orellana Reyes (202210010352)
Carlos Roberto Cardenas Martínez (202430010599)
Jennifer Mariela Marquez Chacon (202330010411)
Michael Abad Diaz Castro (202310040165)
Fernando Abad Diaz Castro (202410040257)
Romel Donaldo Barahona Márquez (202410010359)

REALIZADO POR:
Repositorio: github.com/Kirklen0910/Exa1Pa1199

FECHA:
07 noviembre de 2026

CONTENIDO

1. Introducción
2. Arquitectura y flujo de una petición
3. Organización del proyecto
4. Cliente HTTP con Retrofit y OkHttp
5. Seguridad de red
6. Fotografía y descripción por voz
7. Manejo de estados de la pantalla
8. Pruebas realizadas
9. Evidencias de funcionamiento
10. Registro de peticiones en Logcat
11. Distribución del trabajo
12. Conclusiones

1. INTRODUCCIÓN

En el primer parcial la aplicación guardaba sus registros únicamente en SQLite. En esta tarea la aplicación se transformó en el cliente Android de la Bitácora de campo y ahora trabaja con la API REST desarrollada en la tarea 2.2. Cada registro, compuesto por título, descripción dictada por voz y fotografía, se envía al servidor y se consulta desde él.

El proyecto se organizó con el patrón MVVM y un repositorio, y el consumo de la API se realizó con Retrofit y OkHttp. La aplicación informa al usuario de forma clara los estados de carga, éxito, lista vacía, error del servidor, tiempo de espera agotado y falta de conexión.

2. ARQUITECTURA Y FLUJO DE UNA PETICIÓN

Cada petición recorre las capas en un solo sentido y la respuesta regresa por el mismo camino. La vista nunca llama a Retrofit: únicamente observa el estado que publica el ViewModel mediante LiveData. La Figura 1 muestra el recorrido completo.



Figura 1. Flujo Vista → ViewModel → Repositorio → Retrofit → API

2.1 Ejemplo: creación de un registro

- **Vista.** FormularioRegistroActivity recibe el toque en Guardar y entrega al ViewModel el título, la descripción y la foto.
- **ViewModel.** FormularioRegistroViewModel valida los límites, reduce la imagen, la convierte a Base64 y publica el estado Cargando.
- **Repositorio.** RegistroRepository construye el RegistroRequestDto, invoca RegistroApi y traduce la respuesta o la excepción en un resultado o un ApiError.
- **Retrofit y OkHttp.** Serializan el cuerpo a JSON con Gson y envían POST /api/registros a http://10.0.2.2:5080.
- **API.** Valida los datos, guarda el registro y responde 201 Created; el estado regresa a la vista como Éxito y la lista se actualiza.

3. ORGANIZACIÓN DEL PROYECTO

El código se separó en paquetes según su responsabilidad, como se resume en la Tabla 1.

Tabla 1. Paquetes de la aplicación

Paquete	Clases principales	Responsabilidad
ui	ListaRegistrosActivity, DetalleRegistroActivity, FormularioRegistroActivity, RegistroAdapter, UiHelper	Mostrar datos y recibir eventos del usuario.
viewmodel	ListaRegistrosViewModel, DetalleRegistroViewModel, FormularioRegistroViewModel, UiState	Exponer el estado de cada pantalla con LiveData.
data/remote	RegistroApi, ApiClient, RegistroResumenDto, RegistroDetalleDto, RegistroRequestDto, ProblemDetailsDto	Definir el servicio Retrofit, el cliente OkHttp y los modelos JSON.
data/repository	RegistroRepository, ApiError, RepositorioCallback	Ser el único punto de acceso a la API y traducir errores.
util	ImagenUtils, FechaUtils	Reducir y codificar la foto; formatear fechas.

4. CLIENTE HTTP CON RETROFIT Y OKHTTP

La interfaz RegistroApi declara los cinco endpoints de la API de la tarea 2.2 (Tabla 2).

Tabla 2. Endpoints consumidos por la aplicación

Anotaciones	Método	Endpoint	Respuestas
@GET	listar()	GET /api/registros	200
@GET, @Path	obtener(id)	GET /api/registros/{id}	200, 404
@POST, @Body	crear(dto)	POST /api/registros	201, 400
@PUT, @Path, @Body	actualizar(id, dto)	PUT /api/registros/{id}	204, 400, 404

Anotaciones	Método	Endpoint	Respuestas
@DELETE, @Path	eliminar(id)	DELETE /api/registros/{id}	204, 404

- **Conversión JSON:** GsonConverterFactory.
- **OkHttpClient:** connectTimeout y readTimeout definidos en ApiClient; HttpLoggingInterceptor en nivel BODY, activo solo cuando BuildConfig.DEBUG es verdadero.
- **URL base:** una sola constante, BuildConfig.API_BASE_URL = "http://10.0.2.2:5080/", definida en app/build.gradle.kts. En el emulador, la dirección 10.0.2.2 corresponde a la computadora local.

5. SEGURIDAD DE RED

5.1 Configuración network_security_config

Durante el desarrollo la API se ejecuta con HTTP sin cifrar. Para no permitir este tráfico hacia cualquier servidor, el archivo res/xml/network_security_config.xml lo prohíbe de forma general (cleartextTrafficPermitted="false") y lo habilita únicamente para 10.0.2.2 y localhost. Cualquier otro dominio debe usar HTTPS. En un entorno de producción la API se publicaría con HTTPS y esta excepción se eliminaría.

5.2 Permiso de red local en Android 17

La aplicación usa targetSdk 37 (Android 17). A partir de esta versión, Android bloquea por defecto las conexiones hacia la red local, y la dirección 10.0.2.2 pertenece a ese rango. Durante las pruebas la aplicación mostraba "Tiempo de espera agotado" aunque la API respondía correctamente desde el navegador. El problema se resolvió declarando el permiso ACCESS_LOCAL_NETWORK en el manifiesto y solicitándolo en tiempo de ejecución en ListaRegistrosActivity. Si el usuario lo rechaza, la aplicación explica por qué lo necesita y permite solicitarlo de nuevo.

5.3 Permisos declarados

Tabla 3. Permisos del manifiesto

Permiso	Uso
INTERNET	Comunicación con la API.
CAMERA	Captura de la fotografía.
RECORD_AUDIO	Dictado de la descripción por voz.
ACCESS_LOCAL_NETWORK	Conexión con la API en la red local (Android 17).

6. FOTOGRAFÍA Y DESCRIPCIÓN POR VOZ

- **Fotografía:** se captura con la cámara reutilizando el FileProvider del primer parcial. Antes de convertirla a Base64, ImagenUtils la reduce a un máximo de 1024 px y la comprime en JPEG con calidad 70. En el registro de Logcat del POST el cuerpo completo ocupa alrededor de 16 KB, muy por debajo del límite de 2 MB que valida la API.

- **Voz a texto:** el botón Dictar descripción abre el reconocedor de voz de Android (RecognizerIntent) en español y coloca el texto reconocido en el campo de descripción.
- **Validación en el cliente:** el título debe tener entre 3 y 100 caracteres y la descripción es obligatoria con un máximo de 500, los mismos límites que valida la API. Si el servidor responde 400, sus mensajes se muestran junto al campo correspondiente.

7. MANEJO DE ESTADOS DE LA PANTALLA

Tabla 4. Estados que muestra la aplicación

Estado	Cuándo ocurre	Qué ve el usuario
Cargando	Mientras se espera la respuesta	Barra de progreso visible y botones deshabilitados
Éxito	Respuesta 200, 201 o 204	Datos en pantalla o regreso a la lista
Lista vacía	La API devuelve una lista sin registros	Mensaje "Aún no hay registros" e invitación a crear uno
Error del servidor	Respuesta 4xx o 5xx	Mensaje comprensible; los errores 400 se muestran por campo
Sin conexión	IOException: API apagada o modo avión	Mensaje explicativo y botón Reintentar
Tiempo agotado	SocketTimeoutException	Mensaje explicativo y botón Reintentar

8. PRUEBAS REALIZADAS

Las pruebas se ejecutaron en el emulador Pixel 4 (API 37) con la API de la tarea 2.2 en ejecución local.

Tabla 5. Resumen de pruebas

N.º	Prueba	Resultado esperado	Estado
1	Abrir la aplicación sin registros	Mensaje de lista vacía	Aprobada
2	Crear registro con foto y descripción por voz	Estado Cargando y registro en la lista	Aprobada
3	Listar y actualizar la lista	Registros obtenidos por GET	Aprobada
4	Ver detalle	Foto, título, descripción y fecha	Aprobada
5	Editar registro	Cambios guardados con PUT	Aprobada
6	Eliminar con confirmación	Diálogo y registro eliminado con DELETE	Aprobada
7	Validación de título corto	Error en el campo sin enviar la petición	Aprobada
8	API apagada	Sin conexión y botón Reintentar	Aprobada
9	Modo avión	Sin conexión y botón Reintentar	Aprobada

9. EVIDENCIAS DE FUNCIONAMIENTO

9.1 Operaciones CRUD

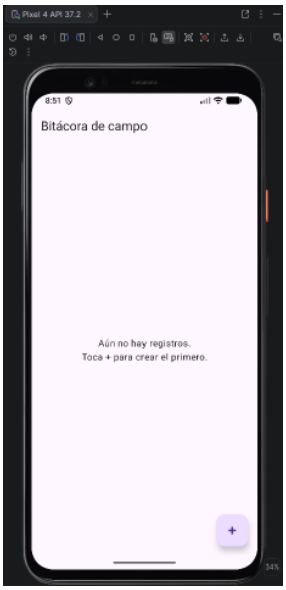


Figura 2. Lista vacía



Figura 3. Creación de un registro (Cargando)



Figura 4. Listado de registros



Figura 5. Detalle con foto, editar y eliminar



Figura 6. Registro editado



Figura 7. Confirmación de eliminación

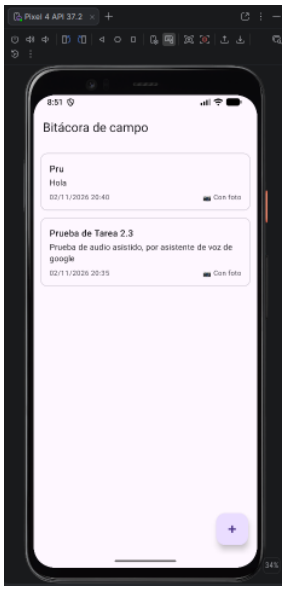


Figura 8. Lista después de eliminar



Figura 9. Error de validación (título corto)

9.2 Errores y falta de conexión



Figura 10. Guardar con la API apagada

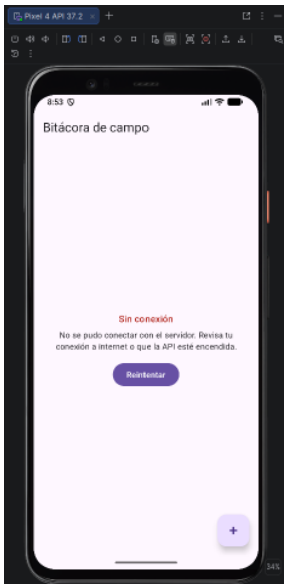


Figura 11. Lista con la API apagada

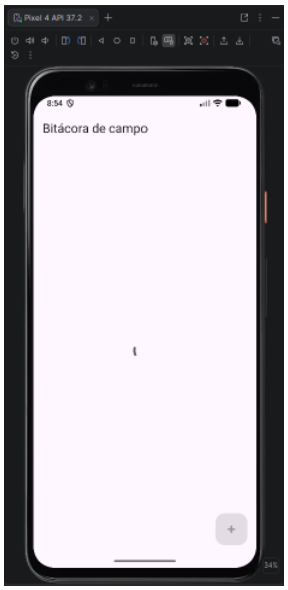


Figura 12. Reintento de conexión

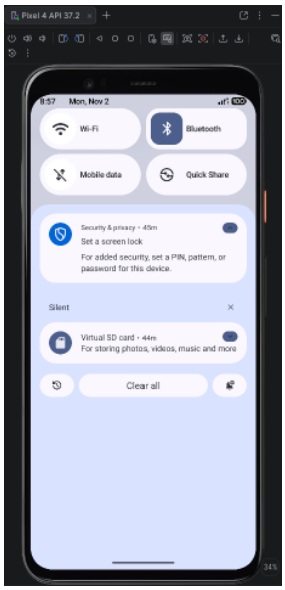


Figura 13. Activación del modo avión

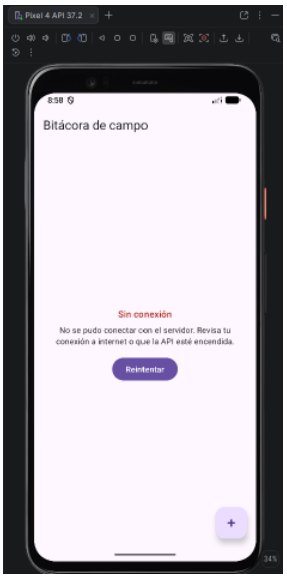


Figura 14. Sin conexión en modo avión

10. REGISTRO DE PETICIONES EN LOGCAT

El interceptor HttpLoggingInterceptor registra en Logcat cada petición y su respuesta (filtro okhttp). Las siguientes capturas evidencian el consumo de los cinco endpoints y el comportamiento con el servidor apagado.


D view not autofillable - has multiline input type
I --> POST http://10.0.2.2:5080/api/registros
I Content-Type: application/json; charset=UTF-8
I Content-Length: 16238
I {"descripcion":"Hola","fotoBase64":"/9j/4AAQSkZJRgABAQAAAQABAAD/4gHYSUNDX1BST0ZJTEUAAQEAAAHIAAAAAAQwAABtbnRyUkdCIFhZWiAH4AA
I AAANöAznQAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAADM4yöRp9jHnKoW+MukafYx5yqGRf/pKvV7AETkAAAAAAAAAAAAAAAAAAA



Figura 19. GET /api/registros/1 con respuesta 200 OK

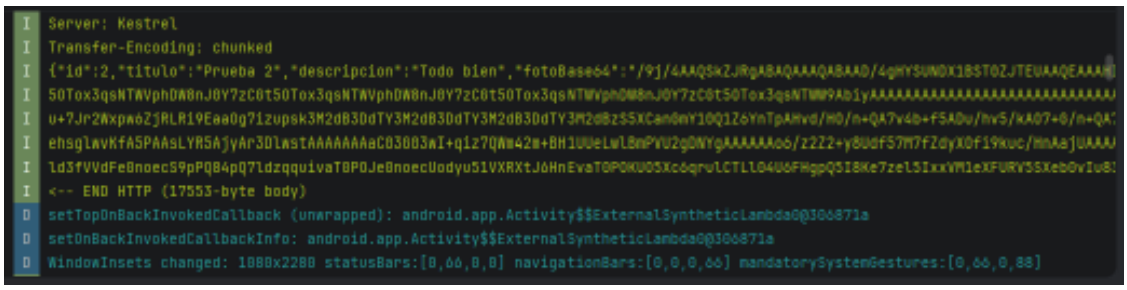


Figura 20. Cuerpo de la respuesta GET con la foto en Base64



Figura 21. DELETE /api/registros/2 con respuesta 204 No Content

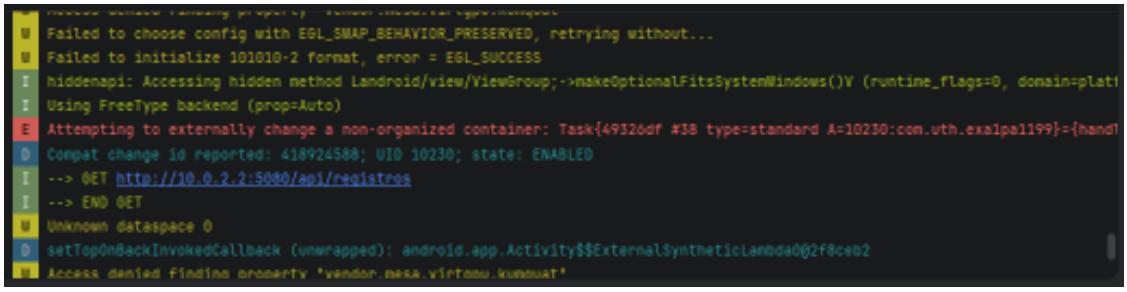


Figura 22. Petición GET sin respuesta con el servidor apagado

11. DISTRIBUCIÓN DEL TRABAJO

Cada integrante desarrolló una capa en su propia rama y la integró a main mediante un Pull Request. El orden de integración garantizó que el proyecto compilara después de cada merge.

Tabla 6. Distribución del trabajo

Integrante	Rama	Aporte
Carlos Roberto Cárdenas Martínez	feature/config-red	Dependencias, URL base en BuildConfig, permisos, network_security_config y README; coordinación, revisión de Pull Requests e informe
[Integrante 2]	feature/data-remote	RegistroApi con los cinco endpoints, ApiClient con timeouts y logging, DTO
[Integrante 3]	feature/data-repository	RegistroRepository y manejo de errores 4xx/5xx, sin conexión y tiempo agotado
[Integrante 4]	feature/viewmodel	ViewModels, UiState, reducción de imagen a Base64 y formato de fechas
[Integrante 5]	feature/ui-formulario	Formulario de creación y edición con cámara, voz a texto y validación
[Integrante 6]	feature/ui-lista-detalle	Lista con RecyclerView, detalle, eliminación con confirmación y documentación del flujo

12. CONCLUSIONES

- La separación en capas con MVVM y un repositorio permitió que las pantallas solo muestren datos, mientras la lógica y el acceso a la API quedan aislados y son más fáciles de probar y mantener.
- Retrofit y OkHttp simplificaron el consumo de los cinco endpoints, y el interceptor de registro facilitó verificar cada petición y respuesta en Logcat.
- Reducir la imagen antes de codificarla en Base64 mantuvo las peticiones livianas y dentro del límite de la API.
- El manejo explícito de los estados mejora la experiencia del usuario, que siempre sabe si la aplicación está cargando, si hubo un error o si no hay conexión.
- La restricción de red local de Android 17 mostró la importancia de conocer los cambios de seguridad de cada versión al apuntar al SDK más reciente.